

Estado atual do pipeline de referência

Snapshot pós-épico "Precisão real" (Sprints 12-16): como os dados brutos viram vetor de features e espectrograma, o que cada classificador recebe, quais algoritmos fazem o trabalho no meio do caminho, e onde as coisas ficaram no fim desta rodada.

● MLP v2 — 4/4 classes $\geq 75\%$ precisão e recall

● CNN — 3/4 classes $\geq 75\%$ precisão e recall

NESTE RELATÓRIO

- | | |
|---------------------------------------|------------------------------------|
| 01 Visão geral do pipeline | 06 Algoritmos por módulo |
| 02 Dataset bruto e pré-processamento | 07 Arquitetura dos classificadores |
| 03 Configuração de canais | 08 Dados de treino oficiais |
| 04 Extração de features — MLP (v1/v2) | 09 Resultado atual |
| 05 Espectrograma — CNN | 10 Pendências e fora do escopo |

01 Visão geral do pipeline

Todo sinal bruto passa por um único ponto de ingestão (normalização por canal) antes de se dividir em **dois caminhos paralelos e independentes** — um alimenta o classificador MLP, o outro a CNN. Os dois são treinados e avaliados separadamente; nenhuma decisão de combiná-los foi tomada.

Sensores brutos

4 canais de vibração (2 acelerômetros biaxiais) + corrente de fase + temperatura (não usada)



Ingestão / normalização

`normalize_ingestion` — física → clip [-1, 1], escala própria por canal



CAMINHO MLP

Decimação ÷32 (corrente) / ÷64 (vibração, 4 canais)



FFT 64 pontos

radix-2 DIF



MDC + `matrix_inv`

f0 refinado (corrente) + espectro de baixa freq.
(vibração) [+ AR via `matrix_inv` na v2]



Vetor de features

129 (v1) ou 133 (v2) valores



MLP

1 cam. oculta (16), ReLU, argmax



Classe

`operacao_normal` · `desbalanceamento` · `desalinhamento` · `desgaste_rolamento`

CAMINHO CNN

Decimação ÷64 (só mancal A)



FFT 64 pontos × 32 janelas

radix-2 DIF, sobrepostas no tempo



Espectrograma 32×32×2

z-score antes do treino



CNN

conv 8×3×3 + pool + densa, argmax

O LMS não está no caminho oficial. `lms/reference.py` existe, está implementado e testado como módulo isolado (8 taps, filtro adaptativo tipo ALE) — mas o vetor de features atual não usa sua saída. Isso é uma divergência deliberada e registrada da retificação 3.5 do enunciado (que pedia "a saída do filtro LMS" no classificador): testes mostraram que a feature derivada do LMS (`e_rms`) piora a separação de `operacao_normal`. Ver `open_decisions.md`.

02 Dataset bruto e pré-processamento

45 fontes de gravação no total ($f_s = 25.600$ Hz), cada uma um ensaio de bancada em uma carga (0/2/4 Nm) e condição de falha. Extração original em Q15 a partir dos arquivos `.mat` / `.tdms`, consolidada num parquet único e dequantizada por `dataset/loader.py` usando fatores de escala medidos na extração.

CLASSE	FONTES	SEVERIDADES	OBSERVAÇÃO
<code>operacao_normal</code>	3	— (1 por carga)	A classe mais escassa do dataset inteiro; motivou o épico "Precisão real"
<code>desbalanceamento</code>	15	5 massas $\times$ 3 cargas	—
<code>desalinhamento</code>	9	3 severidades $\times$ 3 cargas	0,1mm é fisicamente quase indistinguível de normal (Sprint 10.4)
<code>desgaste_rolamento</code>	18	6 severidades (BPFO+BPFI) $\times$ 3 cargas	Classe mais fácil — recall ~99,8% em todo modelo

Ingestão (ponto único de normalização)

`dataset.signal_params.normalize_ingestion` divide cada amostra bruta pela escala do canal e recorta em $[-1, 1]$ — todo módulo a jusante (LMS, FFT, decimação) recebe sinal já nessa faixa. Sem isso, o LMS diverge numericamente (achado da Sprint 11.14).

CANAL	ESCALA
<code>aceleracao_x_mancal_a</code>	18,957
<code>aceleracao_y_mancal_a</code>	28,532
<code>aceleracao_x_mancal_b</code>	8,101
<code>aceleracao_y_mancal_b</code>	7,016
<code>corrente_fase_u</code>	5,521

Divisão treino/teste

Toda avaliação é **agrupada por fonte** — nunca por linha/janela. Uma fonte inteira (uma gravação) vai para um único fold; do contrário, janelas quase-duplicadas da mesma gravação vazariam entre treino e teste.

- **k-fold agrupado (padrão atual, desde a Sprint 11.4):** 3 folds, agrupados por (classe, severidade) — cada severidade aparece em todo fold. É o que gera os números de precisão/recall reportados neste documento.
- **Split fixo treino/val/teste:** legado do Sprint 0, mantido no dataset mas não é mais o caminho usado para reportar métricas (desperdiçava 2/3 das fontes permanentemente em val+teste).

03 Configuração de canais

MLP e CNN usam recortes diferentes dos mesmos 4 canais de vibração — não por limitação técnica, mas porque foi o que cada modelo, testado, sustentou sem overfitting.

MODELO	CANAIS	POR QUÊ
MLP	<code>x_mancal_a</code> , <code>y_mancal_a</code> , <code>x_mancal_b</code> , <code>y_mancal_b</code> (4) + <code>corrente_fase_u</code> (só para f0)	4 canais melhoram o MLP de verdade (Sprint 11.8/11.12) — os 2 acelerômetros biaxiais cobrem os 2 mancais × 2 direções radiais, defensável fisicamente mesmo quando o ganho medido é misto
CNN	<code>x_mancal_a</code> , <code>y_mancal_a</code> (2, "mancal A")	4 canais e um ensemble mancal A+B foram testados (Sprints 11.6/13/16) — ambos pioram <code>operacao_normal</code> por overfitting (mais parâmetros de convolução, mesmas ~45 fontes). Mancal A sozinho é o único ponto estável.

`desalinhamento` na CNN fica com um teto de recall de ~66,7% justamente por essa escolha — a informação que falta está fisicamente no mancal B, mas toda forma testada de incorporá-la desestabiliza as outras 3 classes mais do que resolve essa (detalhe na seção 09).

04 Extração de features — MLP (v1 / v2)

Duas versões vivas em paralelo — decisão explícita de manter as duas, não escolher uma (a decisão de qual vira RTL é separada e posterior).

`features/pipeline.py` — 129 valores efetivamente alimentados ao classificador (130 incluindo `f0_valido`, um flag booleano de diagnóstico que fica salvo no parquet mas é excluído do vetor de entrada do MLP):

- 1 valor `f0_hz` — frequência fundamental do eixo, via corrente ($\div 32$, FFT, MDC + refino parabólico por `matrix_inv`)
- 128 valores espectro de baixa frequência: 32 bins úteis (1-32, $\Delta f_{\text{dec}}=6,25\text{Hz}$) $\times$ 4 canais de vibração, cada um decimado $\div 64$

`BLOCK_SAMPLES = 4096` amostras brutas por bloco (~160ms a 25.600Hz) — imposto pela decimação $\div 64$ precisar de 64×64 amostras para produzir uma janela de 64 pontos decimados.

v2 alternativa oficial / trilha RTL alternativa

`features/pipeline_v2.py` — reaproveita o v1 inteiro sem modificá-lo e adiciona 4 valores: coeficientes AR(4) via Yule-Walker, resolvidos por `matrix_inv`, sobre `x_mancal_a` decimado. **133 valores** no total.

	FEATURES	GANHA EM
v1	129	simplicidade de hardware
v2	133	precisão em toda classe, inclusive <code>operacao_normal</code> , sobre a config oficial de 4 canais

05 Espectrograma — CNN

`cnn/reference.py::build_lowfreq_spectrogram_multichannel` monta uma imagem $32 \times 32 \times 2$ por bloco de sinal — 32 colunas de tempo, cada uma o espectro decimado (bins 1-32, exclui DC) de uma janela deslizando.

- Decimação $\div 64$ (`LOWFREQ_DECIM_FACTOR`) — mesma taxa do `lowfreq_spectrum` do MLP
- Janela/coluna 4.096 amostras brutas, hop de 1.024 amostras entre colunas
- Bloco total `LOWFREQ_BLOCK_SAMPLES = 35.840` amostras brutas (~1,4s a 25.600Hz) por espectrograma inteiro
- Normalização z-score (média/desvio do fold de treino) antes de entrar na rede — sem isso o treino colapsa (Sprint 11.21)

Substituiu a versão nativa original (sem decimação, ~80ms por espectrograma, bins 1-32 de $\Delta f=400\text{Hz}$) — essa enterrava os harmônicos de rotação ($1\times/2\times/3\times$) inteiros dentro do bin DC descartado, deixando a CNN estruturalmente cega à distinção `operacao_normal` / `desbalanceamento` / `desalinhamento` desde a Sprint 7.

06 Algoritmos por módulo

MÓDULO	ALGORITMO	ONDE ENTRA
<code>lms</code>	Filtro adaptativo, 8 taps, tipo <i>adaptive line enhancer</i> (Widrow & Hoff)	Implementado e testado isoladamente — não alimenta o vetor de features oficial (ver nota da seção 01)
<code>fft</code>	Radix-2 DIF, 64 pontos, 6 estágios, saída bit-reversed	MLP (f_0 e espectro de baixa freq.) e CNN (cada coluna do espectrograma)
<code>mdc</code>	MDC de Euclides (GCD por subtração) com fallback de dominância (usa o pico mais forte direto quando ele domina $\geq 5\times$)	Estimativa de <code>f0</code> a partir da corrente — o fallback é o caminho realmente usado neste dataset (GCD genuíno não é confiável, Sprint 6.1/11.2)
<code>matrix_inv</code>	Gauss-Jordan (float)	2 usos concretos: (1) interpolação parabólica do pico de f_0 (sistema 3×3) — v_1 e v_2 ; (2) coeficientes AR(4) via Yule-Walker (sistema 4×4 Toeplitz) — só v_2

Uma terceira formulação de `matrix_inv` (ARX — vibração como resposta à corrente como excitação) foi testada e **descartada**: neutra sobre v_1 , piora v_2 consistentemente (Sprint 11.19).

07 Arquitetura dos classificadores

MLP

Camadas entrada (129 ou 133) $\rightarrow$ 1 oculta (16 neurônios, ReLU) $\rightarrow$ saída (4 classes)

Inicialização `He ( std = sqrt(2/fan_in) )`

Inferência **argmax puro sobre os logits** — sem softmax; bate com o datapath de hardware planejado (`mapping/ml_classifier.md`)

Treino cross-entropy ponderada por classe (`class_weights_from_counts` , esquema `inverse`), gradiente completo (full-batch), lr=0,1, 2000 épocas

CNN

Camadas conv1 (8 filtros 3×3, stride 1, padding 1, ReLU) → max-pool 2×2 → achatado (2048) → densa (4 classes)

Inferência argmax puro sobre os logits, mesma lógica do MLP

Treino mesma ponderação por classe, versão em lote (im2col) por velocidade, lr=0,01, 150 épocas

O peso por classe já foi testado e não deve ser suavizado. Um esquema mais brando (raiz do inverso, teto, ou nenhum peso) foi testado nos dois modelos (Sprint 15) — piorou tudo, com colapso total (recall 0%) na CNN sem peso nenhum. O esquema `inverse` atual é o que sustenta a classe rara, não a causa da precisão baixa.

Metodologia de avaliação/promoção

Para os dois modelos: comparação por k-fold (3 folds, 5 seeds aleatórias por config testada) dá a estimativa honesta de generalização; os pesos **oficiais** salvos vêm de um treino final sobre *todas* as fontes disponíveis (sem retenção), prática padrão depois que a abordagem já foi validada por k-fold.

08 Dados de treino oficiais

O teto de janelas por fonte era artificialmente baixo (Sprint 12) e `operacao_normal` — a classe mais escassa — ganhou um segundo tratamento: janelas sobrepostas (75% de overlap) só para essa classe (Sprint 16), sem tocar nas outras 3.

ARQUIVO	LINHAS/IMGS	OPERACAO_NORMAL	DEMAIS CLASSES
MLP v2 features_dataset_v2_dense_normal.parquet	38.196	3.372 → 13.488 (4× sobreposto)	teto natural (Sprint 12): 11.235 / 6.741 / 6.732

ARQUIVO	LINHAS/IMGS	OPERACAO_NORMAL	DEMAIS CLASSES
CNN spectrogram_dataset_lowfreq_ch0-1_dense_normal.npz	4.328	384 → 1.532 (4× sobreposto)	teto natural: 1.275 / 765 / 756

Sobrepôr *mais de uma classe ao mesmo tempo* foi testado e piora — as classes competem por "capacidade" de fronteira de decisão (Sprint 16). Só `operacao_normal` é densificada.

09 Resultado atual

Meta: precisão e recall $\geq 75\%$ em toda classe. Médias de 5 seeds, k-fold agrupado por fonte.

CLASSE	MLP V2 — PRECISÃO	MLP V2 — RECALL	CNN — PRECISÃO	CNN — RECALL
<code>operacao_normal</code>	87,5%	83,4%	76,4%	99,9%
<code>desbalanceamento</code>	88,4%	94,1%	99,9%	83,0%
<code>desalinhamento</code>	79,5%	78,0%	100%	66,7%
<code>desgaste_rolamento</code>	99,6%	99,8%	~100%	~99,8%

`desalinhamento` na CNN é um **limite estrutural diagnosticado, não uma falha de treino**: a informação que falta está no mancal B (canais não usados pela CNN — seção 03); toda forma testada de incorporá-la (4 canais, ensemble de dois modelos) desestabiliza as outras 3 classes mais do que resolve essa. Não é um alvo de mais ajuste de hiperparâmetro.

10 Pendências e fora do escopo

- **Notebook e relatório de viabilidade** (`analise_smma.ipynb`) — ainda cita conclusões de sprints anteriores ao épico atual; consolidação pendente.

- **Quantização de ponto fixo** — todo módulo de referência (FFT, LMS, matrix_inv, MLP, CNN) roda em float64/"ideal" por enquanto; Q8.8/Q1.15 é etapa seguinte, ainda não iniciada para os módulos de ML.
- **RTL** — épico à parte, replanejamento adiado até a série de precisão fechar (agora fechada). Vira uma sprint por módulo, cada uma com seu próprio testbench.
- **Oversampling/undersampling e focal loss** — cogitados no épico, não executados: não atacam a causa real da precisão baixa (informação faltante do mancal B para desalinhamento na CNN), mesmo diagnóstico já confirmado nas tentativas de canal/ensemble.

Gerado a partir do estado do repositório em 2026-08-30 — código-fonte em [03.Reference/](#), decisões e sprints detalhados em [02.Architecture/](#).